

کارخانه هوشمند

طراحی و شبیه‌سازی سیستم یکپارچه هوشمند صنعتی

پایش بلادرنگ، ایمنی و مدیریت نیروی انسانی توسط برد stm32f103c8



استاد راهنما

جناب دکتر زرگری

تیم پروژه

علیرضا نوید، منتظر نداف مقدم،
امیرحسین اسدی، اشکان پوررستمی، آرین شفیع راد

- معرفی و ضرورت
- اهداف
- معماری سیستم
- زیر بخش های سیستم

۱- پایش محیط و امنیت ۲- کنترل کیفی ۳- مدیریت نیروی کار ۴- امنیت نیروی کار ۵- پایش دستگاه های تولیدی

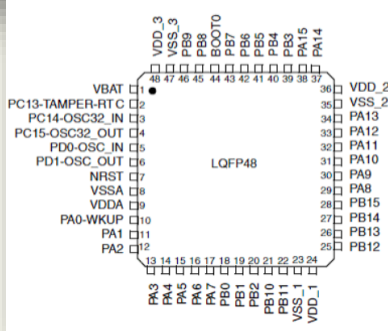
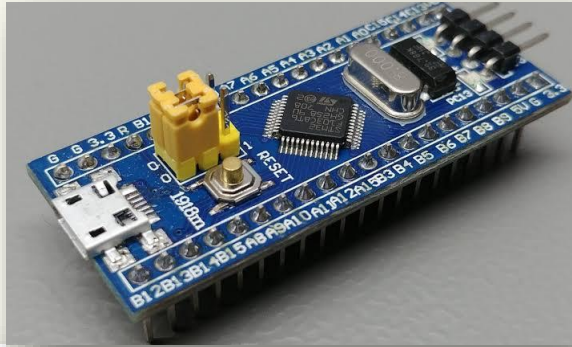
- ارتباط با پایتون
- چالش ها و ادامه توسعه
- نتیجه گیری و منابع

مقدمه و ضرورت

صنعت 4.0؟ کارخانه هوشمند؟!

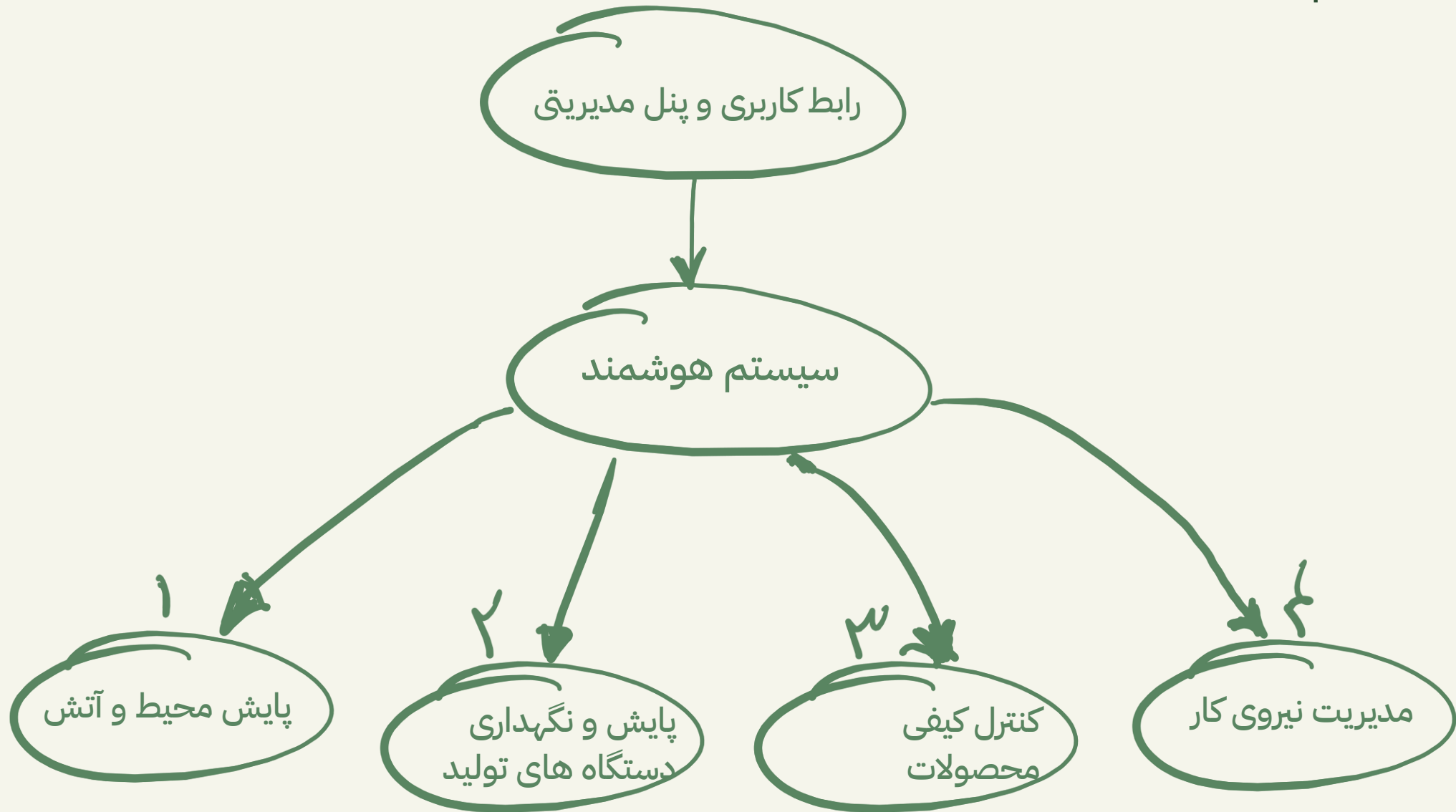
- کارخانه هوشمند یکی از مهم ترین مفاهیم صنعت 4.0 است.
- در این کارخانه، حسگر ها و سیستم های نهفته برای پایش محیط و کنترل تجهیزات به کار می‌روند.
- هدف، افزایش بهره وری، کاهش خطا و ارتقای ایمنی محیط صنعتی است.
- استفاده هوشمندانه از سیستم های خبره موجب افزایش کیفیت محصولات تولیدی است.
- افزایش نیاز صنایع به اتوماسیون و نظارت لحظه ای
- کاهش خطا های انسانی و توقف خطوط تولید افزایش ایمنی کارکنان و تجهیزات





- طراحی و شبیه سازی یک کارخانه هوشمند مبتنی بر سیستم های نهفته
- پیاده سازی سیستم پایش و کنترل لحظه ای تجهیزات صنعتی
- افزایش ایمنی محیط کارخانه با تشخیص حریق، دود و شرایط غیرعادی
- نظارت بر سلامت و عملکرد ماشین آلات و کاهش احتمال خرابی
- مدیریت فرآیند تولید و ثبت وضعیت بخش های مختلف کارخانه
- نمایش اطلاعات و هشدار ها به صورت لحظه ای در داشبورد مدیریتی

توسط میکروکنترلر STM32f103c8، سنسورها و ارتباط بین اجزای سیستم و شبیه سازی شماتیکی عملکرد یک کارخانه و خط تولید هوشمند در محیط Proteus



• این پروژه به ۴ بخش

۱- پایش محیط و امنیت

۲- کنترل کیفی

۳- پایش دستگاه های تولیدی

۴- مدیریت نیروی کار و امنیت نیروی کار

تقسیم شده که هر بخش بطور مجزا توسط یک

نفر از اعضا پیاده سازی شده است.

• سعی شده تا معماری به صورت لایه ای و

مدولار طراحی شده است. این طراحی باعث

می شود هر زیر سیستم به طور مستقل توسعه

یابد اما تمام بخش ها از طریق سیستم مرکزی

با یکدیگر هماهنگ باشند.

• معماری این پروژه بر اساس الگوی سه لایه ای:

۱- فریمور و سخت افزار

۲- سیستم مرکزی

۳- رابط کاربری

سامانه پایش محیط و ایمنی حریق STM32

کنترل هوشمند دما، اعلام حریق و ارسال تله متری

سیستم های نهفته با درایور های HAL و معماری دو برده

زیر بخش ۱؛ معماری و ساختار فایل ها



فایل های (adc, gpio, usart, rtc) HAL

پیکربندی سخت افزاری میکروکنترلر شامل ADC1 برای خواندن سنسور LM35، GPIO ها برای ورودی/خروجی ها، UART برای تله متری و RTC برای ثبت زمان واقعی



lcd_parallel.c / .h

درایور نمایشگر متنی 20 در 4 به صورت Parallel 4-bit، توابع مدیریت کرسور، ارسال دستور/داده و تولید کاراکتر انیمیشن شعله

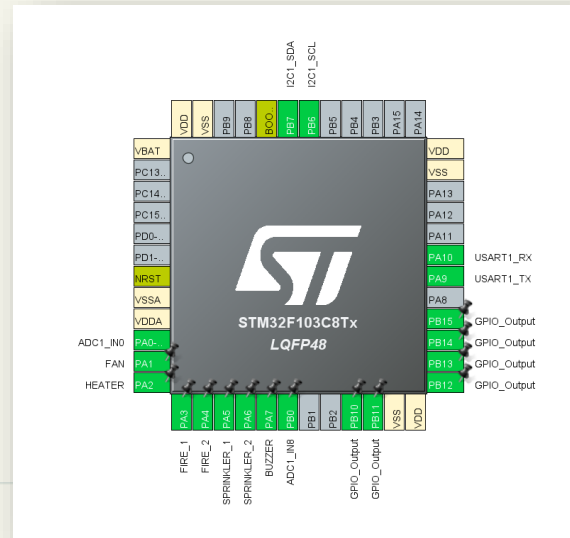


environment.c / .h

ماژول اصلی سیستم شامل الگوریتم فیلتر میانگین متحرک دما، کنترل اکچوئتر ها (فن و هیتر)، مدیریت ایمنی حریق و پردازش دستورات ورودی UART

زیر بخش ۱؛ منطق سخت افزاری و پین ها

توضیحات و قطعه متصل شده	نوع عملکرد	نام سیگنال	پورت / پین
اتصال به سنسور دمای آنالوگ LM35 (مقیاس ۳۳۰ درجه / ۴۰۹۵)	ورودی آنالوگ	ADC1_IN0	PA0
فرمان به سیستم سرمایشی (فن) و گرمایشی (هیتر)	خروجی دیجیتال	PIN_FAN / PIN_HEATER	PA1 / PA2
اتصال به سنسور های تشخیص حریق (Pull-down داخلی)	ورودی دیجیتال	PIN_FIRE_1 / PIN_FIRE_2	PA3 / PA4
کنترل اسپرینکلر های ۱ و ۲ و بازار هشدار صوتی حریق	خروجی دیجیتال	SPRINKLER & BUZZER	PA5 / PA6 / PA7
پین های کنترلی E , RS و خطوط داده D4..D7 نمایشگر LCD 20x4	خروجی دیجیتال	LCD Parallel Bus	PB10 .. PB15



زیر بخش ۱؛ منطق عملکرد و الگوریتم های اجرایی



۱. خواندن و فیلترسازی دما

- نمونه برداری از سنسور:
سیگنال آنالوگ سنسور دما از طریق ADC پایه PA0 / ADC1_IN0 خوانده می شود. مقدار خام ADC (بین ۰ تا ۴۰۹۵ در مد ۱۲ بیتی) دریافت و با فرمول خطی به ولتاژ و سپس به درجه سانتی گراد تبدیل می شود.
- بافر چرخشی:
برای کاهش نویز نمونه برداری آنالوگ و جلوگیری از نوسان ناگهانی عملگر ها، از الگوریتم میانگین متحرک استفاده می شود.
- آخرین نمونه های دما در یک بافر چرخشی با ظرفیت مشخص ذخیره می شوند.
- در هر دور اجرای حلقه، نمونه جدید جایگزین قدیمی ترین نمونه شده و میانگین کل نمونه های بافر به عنوان دمای معتبر سیستم محاسبه می گردد.



۲. منطق کنترل دما و عملگر سرمایشی/گرمایشی

- مقایسه با آستانه تولرانس:
سیستم دمای میانگین محاسبه شده را با دمای هدف و درصد تولرانس مجاز مقایسه می کند.
- تصمیم گیری برای عملگر ها:
اگر دمای میانگین < حد بالا: فن PIN_FAN روی PA1 روشن و هیتر خاموش می شود.
- اگر دمای میانگین > حد پایین: هیتر PIN_HEATER روی PA2 روشن و فن خاموش می شود.
- اگر دما در محدوده امن باشد: هر دو عملگر فن و هیتر خاموش می مانند.

زیر بخش ۱؛ منطق عملکرد و الگوریتم های اجرایی



۳. منطق ایمنی حریق و قفل اضطراری

- تشخیص سنسور های اعلام حریق:
ورودی دو سنسور حریق PIN_FIRE_1 و PIN_FIRE_2 روی پایه های PA3 و PA4 به صورت دیجیتال پایش می شود. فعال سازی حالت قفل اضطراری: در صورت فعال شدن هر یک از سنسور ها، سیستم بلافاصله وارد وضعیت اضطراری می شود: آب پاش مربوط SPRINKER_1 یا SPRINKLER_2 فعال می گردد.
بوق هشدار به صورت چشمک زن/تناوبی به صدا در می آید. کلیه عملگر های اقلیمی (فن و هیتر) برای جلوگیری از تغذیه یا گسترش حریق غیر فعال می گردند.
- زمان بندی قفل و بازگشت خودکار: سیستم زمان آخرین تشخیص حریق را نگه می دارد پس از قطع سیگنال سنسور ها و گذشت یک بازه زمانی مشخص ALARM_LOCK_DURATION_MS، قفل ایمنی به صورت خودکار آزاد شده و کنترل اقلیمی به حالت عادی باز می گردد.



۴. معماری ارتباطی و پروتکل تله متری

- ارسال بسته های تله متری در هر ثانیه، برد اصلی یک بسته متنی ساختار یافته (حاوی زمان RTC، تاریخ، دمای لحظه ای، دمای میانگین، وضعیت حریق، وضعیت قفل و دمای هدف) را از طریق UART1 به برد ثانویه ارسال می کند.
نمونه فرمت بسته:
@TIME=hh:mm:ss|DATE=yy-mm-dd|CUR=xx.x|AVG=xx.x|FIRE=x|LOCK=x|TARG=xx.x
- پردازش دستورات دریافتی:
دریافت کاراکتر ها به صورت غیر بلاک کننده (Interrupt Mod) انجام می شود. پس از دریافت کاراکتر پایان خط \r یا \n، دستوراتی مانند تنظیم دمای هدف set_temp، تنظیم ساعت و تاریخ RTC (set_time / set_date)، بی صدا کردن آلارم یا ریست سیستم پارس و اعمال می گردند.

زیر بخش ۱؛ توابع کلیدی و کاربردی پروژه (برد اصلی و ثانویه)

توابع برد ثانویه

`void SEC_Init(UART_HandleTypeDef*, LCD_HandleTypeDef*)`

راه اندازی ماژول برد ثانویه، تنظیم وقفه دریافت UART و آماده سازی صفحه مانیتورینگ.

`void SEC_UART_RxCpltCallback(UART_HandleTypeDef *huart)`

دریافت غیربلاک کننده (Interrupt) داده تله متری، بررسی صحت بسته و ذخیره در بافر.

`void SEC_ParseTelemetry(uint8_t *rx_buffer)`

تجزیه بسته داده دریافتی، استخراج دما و وضعیت ایمنی و استخراج آلارم ها جهت نمایش.

`void SEC_SendConfigCommand(uint8_t cmd_id, float param)`

ارسال دستورات تنظیم مجدد مانند تغییر دمای هدف یا حد مجاز حریق به برد اصلی.

توابع برد اولیه اصلی

`void ENV_Init(ADC_HandleTypeDef*, UART_HandleTypeDef*, RTC_HandleTypeDef*)`

مقدار دهی اولیه ماژول، تنظیم اشاره گر های HAL، راه اندازی سنسور ها و LCD

`void ENV_Task(void)`

حلقه پردازش اصلی: فراخوانی فیلتر، کنترل اقلیم، مدیریت حریق و بروزرسانی نمایشگر.

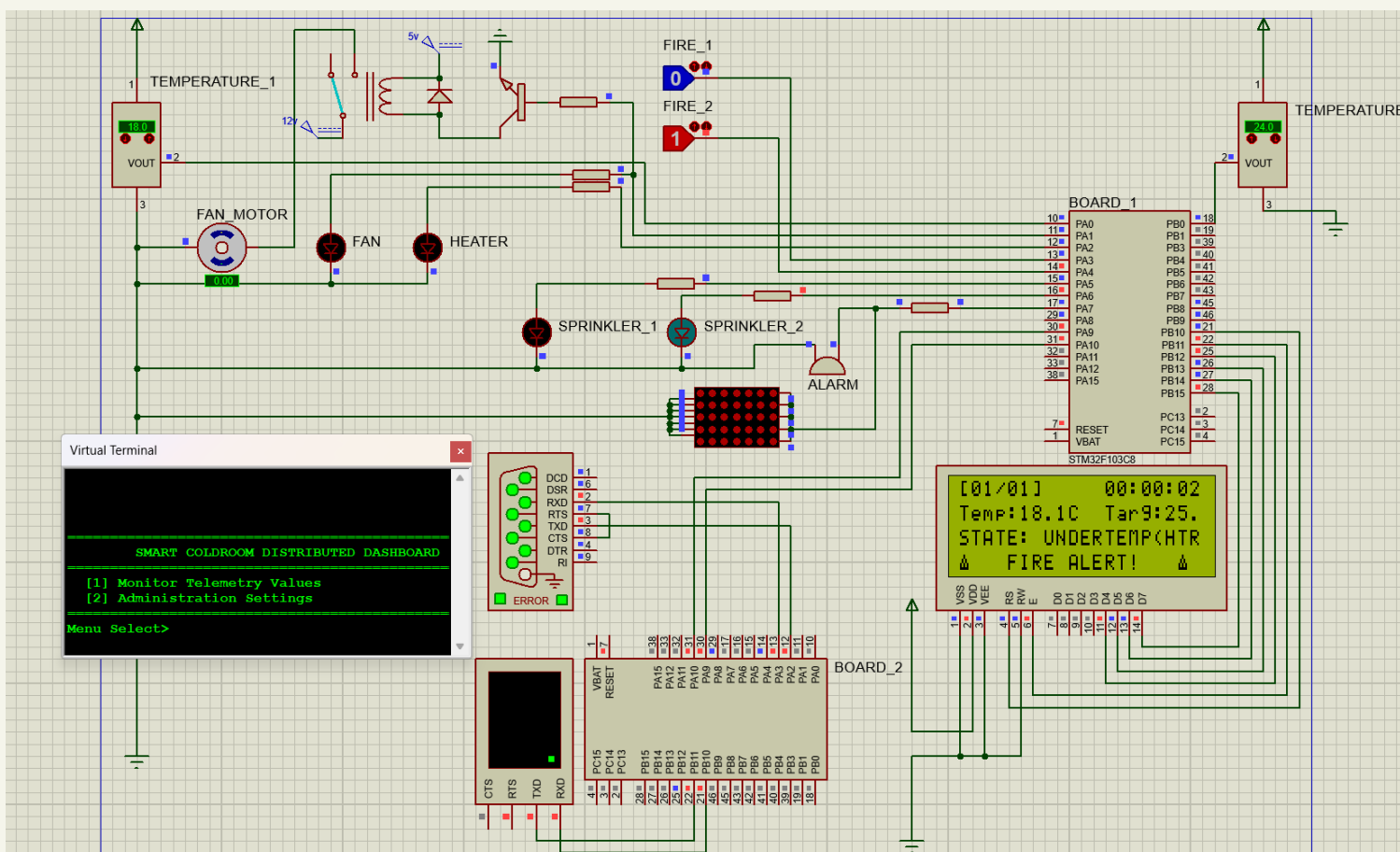
`float ENV_ProcessADC_Filter(void)`

نمونه برداری آنالوگ LM35 و محاسبه میانگین متحرک (Moving Average) برای حذف نویز.

`void ENV_SendTelemetry(void)`

قالب بندی و ارسال بسته تله متری (دما، زمان RTC، وضعیت سیستم از طریق UART)

زیر بخش ۱؛ شبیه سازی و پیاده سازی سخت افزاری



- میکروکنترلر STM32F103 : هسته های اصلی پردازش و اجرای برنامه دو برده.
- نمایشگر LCD 20x4 : نمایش زمان واقعی، دمای لحظه ای/میانگین، وضعیت اقلیم و هشدار متحرک حریق
- سنسور دما LM35 : متصل به پایه PA0 (ADC Channel 0) جهت سنجش دقیق دما.
- عملگرها: فن، هیتر، بازر صوتی و اسپرینکلرهای اطفای حریق متصل به پورت
- ارتباط سریالی دو برده: شبیه سازی ارتباط UART بین برد اصلی و ثانویه جهت ارسال تله متری و دریافت تنظیمات.

سامانه پایش کنترل کیفی STM32

کنترل هوشمند محصولات تولید شده ، تفکیک و گزارش

سیستم‌های نهفته با درایورهای HAL



زیر بخش ۲؛ معماری و ساختار فایل ها



فایل های (adc, gpio, usart) HAL

پیکربندی سخت افزاری میکروکنترلر شامل ADC1 برای خواندن سنسور های آنالوگ مانند پتانسیومتر ، GPIO ها برای ورودی/خروجی ها، UART برای گزارش و دریافت مقادیر سنسور ها RTC برای ثبت زمان محصولات



lcd_parallel.c / .h

درایور نمایشگر متنی 20 در 4 به صورت Parallel 4-bit، توابع مدیریت کرسور، ارسال دستور/داده و برای گزارش وضعیت محصول

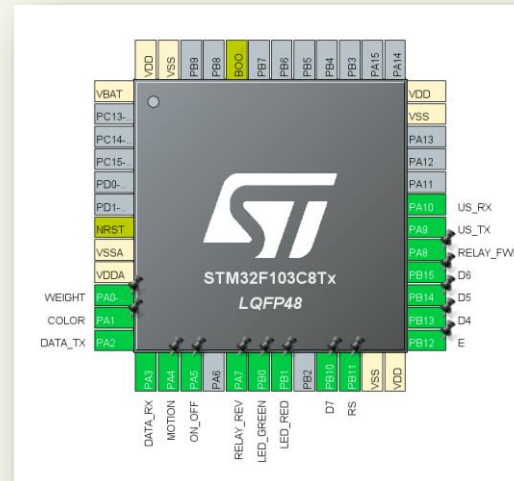


Quality.c / .h

تشخیص حرکت و سپس دریافت مقادیر مربوط به پارامتر های صحت سنجی محصول و بررسی پارامتر ها و در آخر تفکیک محصول تأیید نشده

زیر بخش ۲؛ منطق سخت افزاری و پین ها

توضیحات و قطعه متصل شده	نوع عملکرد	نام سیگنال	پورت /پین
اتصال به وزن سنج و با تمام سنسور های آنالوگ	ورودی آنالوگ	WEIGHT	PA0
کنترل LED ها و موتور های نوار نقاله	خروجی دیجیتال	LED_RED, GREEN/ RELAY_REV, FWD	PB0/PB1 PA7, PA8
اتصال به سنسور های دیجیتال مثل رنگ ، حرکت و دکمه خاموش و روشن	ورودی دیجیتال	COLOR / MOTION / ON_OFF	PA1/ PA4 / PA5
ارتباطات UART برای ارسال گزارش و دریافت مقادیر سنسور	خروجی و ورودی دیجیتال	DATA_TX / DATA_RX US_TX / US_RX	USART1&2
پین های کنترلی RS ,E و خطوط داده D4..D7 نمایشگر LCD 20x4	خروجی دیجیتال	RS / E / D4..D7	PB10 .. PB15



زیر بخش ۲؛ منطق عملکرد و الگوریتم های اجرایی

۱. مراحل اولیه

- روشن و خاموش شدن : سیستم یک دکمه روشن و خاموش دارد برای مواقعی که کنترل کیفیت لزومی ندارد و نوار نقاله محصولات را بدون توجه به کیفیت انتقال دهد پس برای فعال شدن لازم است ابتدا سیستم روشن شود
- تشخیص حرکت : حتی بعد از روشن شدن سیستم ممکن است سرعت تولید محصول کم باشد و سیستم مدام مقادیر را چک کند بنابراین یک دکمه به نمایندگی سنسور حرکت وجود دارد تا اگر حرکت تشخیص داده شد سیستم شروع به کار کند
- نحوه تعریف : این دو پایه به صورت interrupt تعریف شدند تا سیستم زیاد شلوغ نباشد و فقط وقتی سیستم روشن است و حرکت تشخیص داده شد نمونه برداری شود

۲. خواندن مقادیر

- نمونه برداری از سنسور: مقدار آنالوگ از پتانسیومتر (نماینده سنسور های آنالوگ) خوانده می شود (بعد از تشخیص حرکت) و با سرعت 55.5 سایکل نمونه برداری میشود و بنابر سنسور مورد استفاده و data sheet مقیاس بندی می شود
- تشخیص رنگ: بجای سنسور رنگ از لاجیک استیت استفاده کردیم و مقدار آن را می خوانیم
- تشخیص طول: از سنسور اولتراسونیک برای تشخیص طول محصول نهایی استفاده می کنیم که با پروتکل UART کار می کند.

زیر بخش ۲؛ منطق عملکرد و الگوریتم های اجرایی

۳. منطق کنترل کیفیت محصول

- مقایسه مقدار آنالوگ با بازه مقدار مطلوب:
مقدار خوانده شده از سنسور را بعد از تبدیل به دیجیتال با مقدار مطلوبی که در کد وجود دارد (از کنترلر main دریافت می شود) مقایسه کرده و مقدار یک مقدار صفر یا یک مشخص می کند که قطعه وزن مطلوبی دارد یا نه
- مقدار های دیجیتال یا با پروتکل :
مقداری که فقط از سنسور معمولی دیجیتال می آید را با مقدار مطلوب (صفر یا یک) مقایسه می کنیم و در نتیجه خودش یا نقیضش وارد شرط میشود و سنسور های با پروتکل مثل موارد آنالوگ با بازه مقایسه می شود

۴. کنترل عملگر ها

- دو LED رشته ای بزرگ وجود دارد برای مشخص کردن صحت بیا خرابی محصول که اگر محصول سالم بود مدتی چراغ سبز و اگر خراب بود مدتی چراغ قرمز روشن می شود تا اعلام وضعیت فیزیکی کند و نتیجه را با مقدار های سنسور ها (مثلا قطعه سالم به فرم رشته وزن: رنگ : طول :) توسط UART به برنامه مدیریت و ترمینال مجازی گزارش بدهد البته یک آرایه به طول 20 وجود دارد که وضعیت 20 محصول اخر را نگه می دارد که توسط UART به ترمینال مجازی و برنامه مدیریت گزارش می دهد (مقادیر کل و مقادیر سالم و خراب نیز فرستاده میشود)
- و در LCD وضعیت هر قطعه را گزارش می دهد توسط ارتباط 4-bit در نهایت اگر قطعه خراب بود رله رو فعال می کند تا موتور DC جهت نوار نقاله را بچرخاند تا محصول خراب جدا شود سپس بعد از مدت زمان معلوم (که به سرعت نوار نقاله بستگی دارد) رله دوم فعال می شود تا مسیر را به جای اصلی خود برگرداند

زیر بخش ۲؛ شبیه سازی و پیاده سازی سخت افزاری

- میکروکنترلر STM32F103 : هسته های اصلی پردازش و اجرای برنامه تک برده.
- نمایشگر LCD 20x4 : نمایش زمان شروع سیستم ، وضعیت آخرین محصول رد شده، تعداد کل، تعداد محصولات سالم و خراب
- سنسور فاصله سنج Ultra Sonic: مقدار طول محصول رد شده
- عملگرها: LED های رشته ای سبز و قرمز و رله های کنترل کننده ماتور DC
- ارتباط UART: برای گزارش وضعیت به ترمینال مجازی و برنامه اصلی .

ماژول مدیریت اطلاعات و نظارت بر کارگران STM32

مدیریت تردد، ثبت درخواست ها و داشبورد تعاملی سرکارگر

پایه سازی نرم افزاری بر پایه ماشین وضعیت (State Machine) و شبکه Dual-UART

زیر بخش ۳؛ معماری نرم افزار و ماشین وضعیت



فایل های لایه HAL (usart, gpio)

پیکربندی سخت افزاری میکروکنترلر شامل UART1 برای ارتباط با ترمینال مجازی کارگران، UART3 برای تبادل اطلاعات با پایگاه داده مرکزی و GPIO برای تایید سخت افزاری مرخصی ها.



UART_Handler / Buffer Control

مدیریت وقفه های سریال
(HAL_UART_RxCpltCallback) به صورت Non-blocking. پیاده سازی بافر های دریافت، مدیریت کلید های اصلاحی (مانند Backspace) و تفکیک پیام های منو از دیتا.

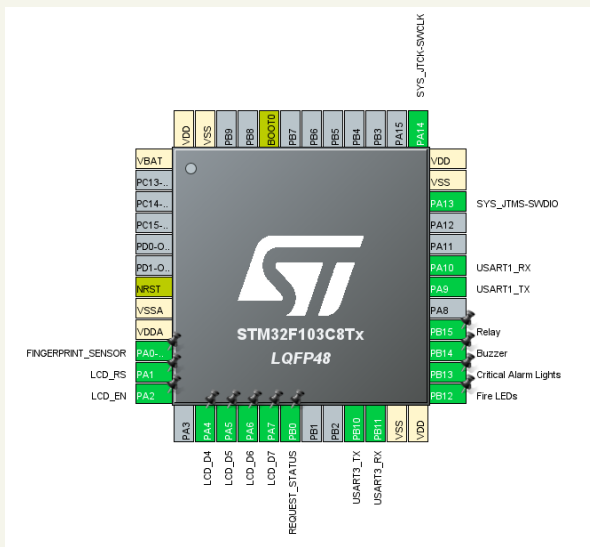


main.c / State Machine

پیاده سازی هسته منطقی رابط کاربری (UI) مبتنی بر Enum. مدیریت یکپارچه وضعیت های سیستم و پردازش بلادرنگ دستورات و منو های ترمینال.

زیر بخش ۳؛ منطق سخت افزاری و پین‌ها

توضیحات و قطعه متصل شده	نوع عملکرد	نام سیگنال	پورت /پین
اتصال به ترمینال مجازی (Virtual Terminal) جهت تعامل متنی با کاربر، دریافت کلید ها و نمایش منو ها	ارتباط سریال	USART1_TX / USART1_RX	PA9 / PA10
ارتباط ضربدری با ماژول COMPIM برای اتصال به پایگاه داده مرکزی و ثبت رکورد ها	ارتباط سریال	USART3_TX / USART3_RX	PB10 / PB11
شبیه سازی سخت افزاری کلید فیزیکی برای تایید یا رد درخواست مرخصی کارگران توسط مدیر	ورودی دیجیتال	VACATION_CONFIRM	PB0
ارسال پیام های وضعیت و اطلاعات هویتی کارگر به نمایشگر کاراکتری (LCD) جانبی	خروجی دیجیتال	LCD_RS, EN, D4..D7	پین‌های آزاد GPIO



زیر بخش ۳؛ منطق عملکرد و الگوریتم‌های اجرایی

۱. مدیریت تردد و پنل کارگران

- احراز هویت بلادرنک:
دریافت شناسه (ID) از طریق ترمینال متنی و ارسال آن به پایگاه داده مرکزی جهت صحت سنجی و استخراج نام کارگر.
- ثبت ورود و خروج (Clock-in/out):
ثبت دقیق زمان شروع و پایان شیفت کاری و پیاده سازی مکانیزم جلوگیری از ثبت رکورد های تکراری در پایگاه داده.
- یکپارچه سازی رابط کاربری با سخت افزار؛ بررسی وضعیت تایید یا رد مرخصی از طریق پایش لحظه ای وضعیت پین های ورودی میکروکنترلر (مانند پین PBO).

۲. نظارت مدیریتی و داشبورد اطلاعات

- لاگین امن و کنترل دسترسی:
تغییر وضعیت State سیستم به پنل مدیریتی تنها در صورت تایید نام کاربری و رمز عبور، جهت جلوگیری از دسترسی افراد غیرمجاز.
- مشاهده داشبورد یکپارچه:
امکان وارد کردن ID کارگر توسط مدیر، فراخوانی اطلاعات هویتی و وضعیتی او از پایگاه داده و نمایش خط به خط آن روی ترمینال.
- مداخله مستقیم در نیروی کار:
اعمال دسترسی های ویژه برای سرکارگر شامل: ثبت دستی غیبت (Register Absence)، خروج اجباری کارگران و فرمان توقف خط تولید در شرایط اضطراری.

زیر بخش ۳؛ منطق عملکرد و الگوریتم های اجرایی

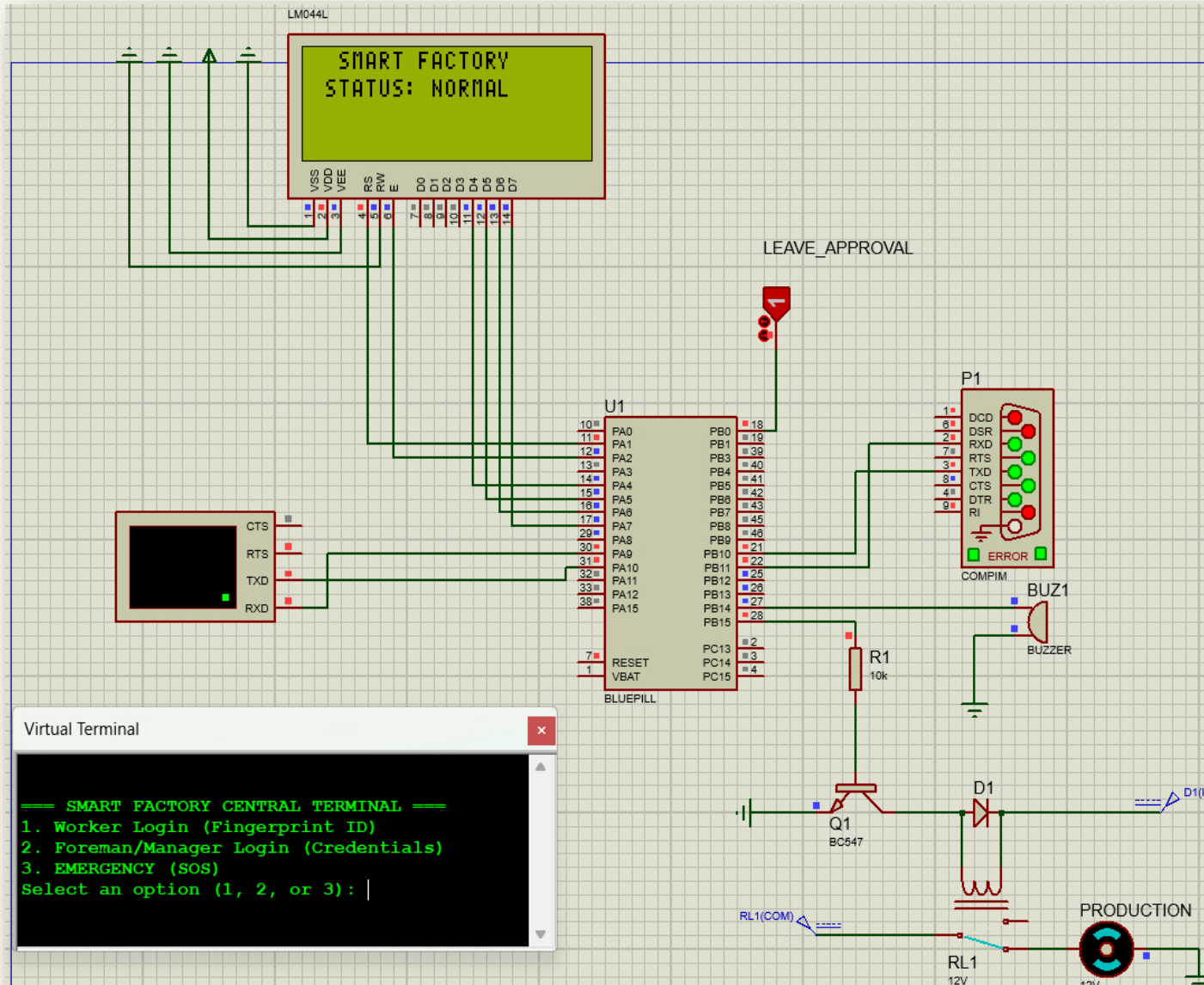
۳. معماری ارتباطی و پردازش پیام ها

- ارتباط دوگانه (Dual-UART):
تفکیک کامل پورت مربوط به ترمینال مجازی (کاربر) از پورت متصل به پایگاه داده مرکزی، تا ترافیک داده ها باعث کندی سیستم نشود.
- پروتکل تفکیک دیتا:
استفاده از پیشوندهای اختصاصی (مانند @MSG و @PRT) از سمت سرور، تا میکروکنترلر بتواند منو های رابط کاربری را از داده های خام داشبورد تشخیص داده و از تداخل متون جلوگیری کند.

۴. مدیریت رویدادها و پایداری سیستم

- دریافت غیربلاک کننده (Non-blocking):
دریافت کاراکتر به کاراکتر دستورات با استفاده از وقفه ها (Interrupts)، تا پردازنده میکروکنترلر هرگز در حلقه های انتظار قفل نشود.
- محافظت در برابر خطا (Error Handling):
پیاده سازی مکانیزم قفل بافر و مدیریت خطای سخت افزاری. این کار از هنگ کردن سیستم (خطای سرریز داده یا Overrun) در زمانی که کاربر سریع تایپ می کند یا اطلاعات حجیمی از دیتابیس دریافت می شود، جلوگیری می کند.

زیر بخش ۳؛ شبیه سازی و پیاده سازی سخت افزاری



- میکروکنترلر STM32F103: پردازنده مرکزی جهت اجرای ماشین وضعیت (State Machine) و مدیریت بلادرنگ پایانه های ارتباطی.
- ترمینال مجازی (Virtual Terminal): شبیه سازی رابط کاربری تعاملی (CLI) برای ورود اطلاعات کارگران، احراز هویت و نمایش داشبورد های مدیریتی.
- ماژول ارتباطی (COMPIM): درگاه فیزیکی متصل به UART3 جهت برقراری ارتباط پایدار و دو طرفه بین میکروکنترلر و پایگاه داده مرکزی سیستم.
- ورودی های کنترلی (GPIO): پایش پین های میکروکنترلر (مانند PB0) برای شبیه سازی کلید های فیزیکی جهت تایید یا رد درخواست ها توسط سرکارگر.
- نمایشگر وضعیت (LCD 16x2): نمایشگر جانبی متصل به پایه های خروجی، جهت اعلام سریع وضعیت ورود/خروج و هشدار های سیستمی به صورت دیداری.

سامانه ایمنی و نظارت کارگران

حفاظت فردی و مدیریت خطاهای محیطی در کارخانه هوشمند

توسعه یافته به صورت ماژولار و مبتنی بر معماری رویداد گرا



زیر بخش ۴؛ معماری نرم افزار و ماشین وضعیت



سیستم مرکزی و پروتکل Freeze

پیاده سازی سیستم مرکزی برای مدیریت فایل ها، پاداش ها و بررسی توانایی فیزیکی/سنی کارگران. ارسال پکت `CMD_TIMER:FREEZE\r\n` به سرور در زمان توقف ایمنی و ارسال پکت `RESUME` پس از رفع خطا.



مدیریت اکشن های ایمنی

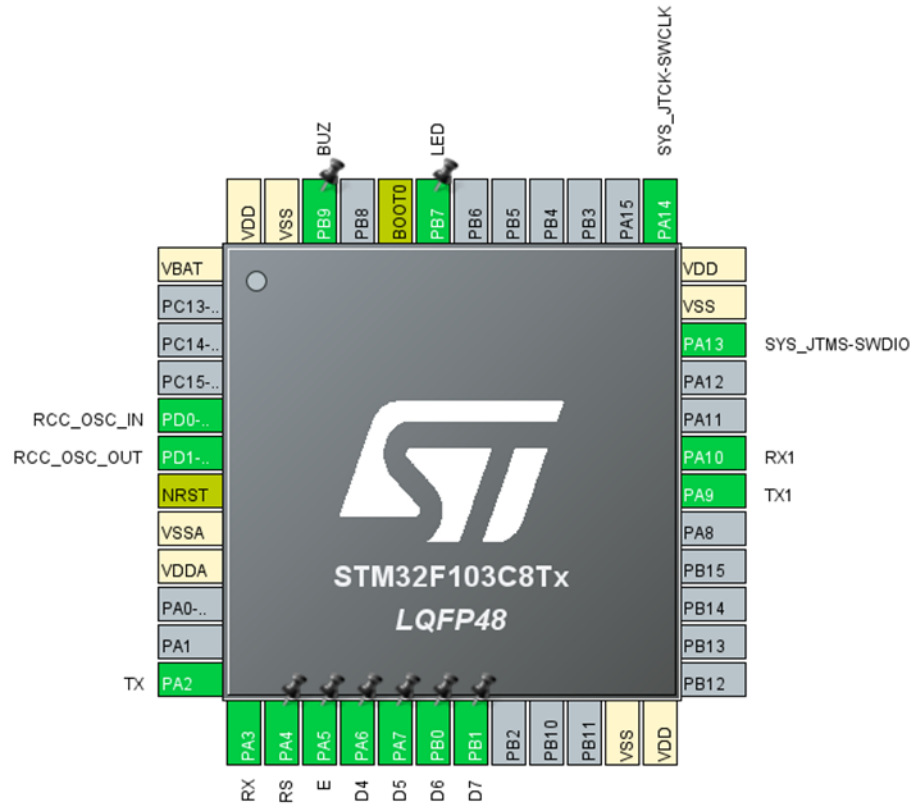
فعال شدن پرچم `is_errrr_active` به محض تشخیص خطا، روشن شدن آژیر و چراغ خطر در کسری از ثانیه و تغییر وضعیت نمایشگر به حالت قرمز.



پیاده سازی هسته ارتباطی و پردازش خطا

راه اندازی وقفه های دریافت سریال (UART Interrupts) برای بیدار شدن سریع میکروکنترلر و توسعه تابع `Process_Terminal_Command` جهت تحلیل رشته های ورودی مربوط به خطا های `CLOTH` و `DISTANCE`

زیر بخش ۴؛ منطق سخت افزاری و پین‌ها



توضیحات و قطعه متصل شده	عملکرد	پورت / پین
LED	هشدار بصری	PB7
آژیر	هشدار صوتی	PB9
نمایشگر	خروجی دیجیتال	A4 .. A7 / B0 / B1
سیستم مرکزی (COMPIM)	ورودی و خروجی دیجیتال	A2 / A3
ترمینال مجازی	ورودی و خروجی دیجیتال	A9 / A10
-	پیکربندی پایه سیستمی	PA13 / PA14 / PD0 / PD1

زیر بخش ۴؛ منطق عملکرد و الگوریتم های اجرایی

۱. نظارت مستقیم بر ایمنی فیزیکی

- نظارت لحظه ای بر رعایت پروتکل های ایمنی از طریق بررسی وضعیت لباس کارگران
- بررسی و حفظ فاصله ایمنی کارگران از دستگاه های خطرناک جهت جلوگیری از حوادثی نظیر قطع عضو به دلیل حواس پرتی

۲. نظارت غیر مستقیم و مدیریت کارها

- بررسی مناسب بودن تسک های محوله نسبت به محدوده سنی کارگر
- کمک به افراد مسن تر کارخانه در جهت امرار معاش راحت تر و جلوگیری از آسیب های فیزیکی در طولانی مدت

زیر بخش ۴؛ منطق عملکرد و الگوریتم‌های اجرایی

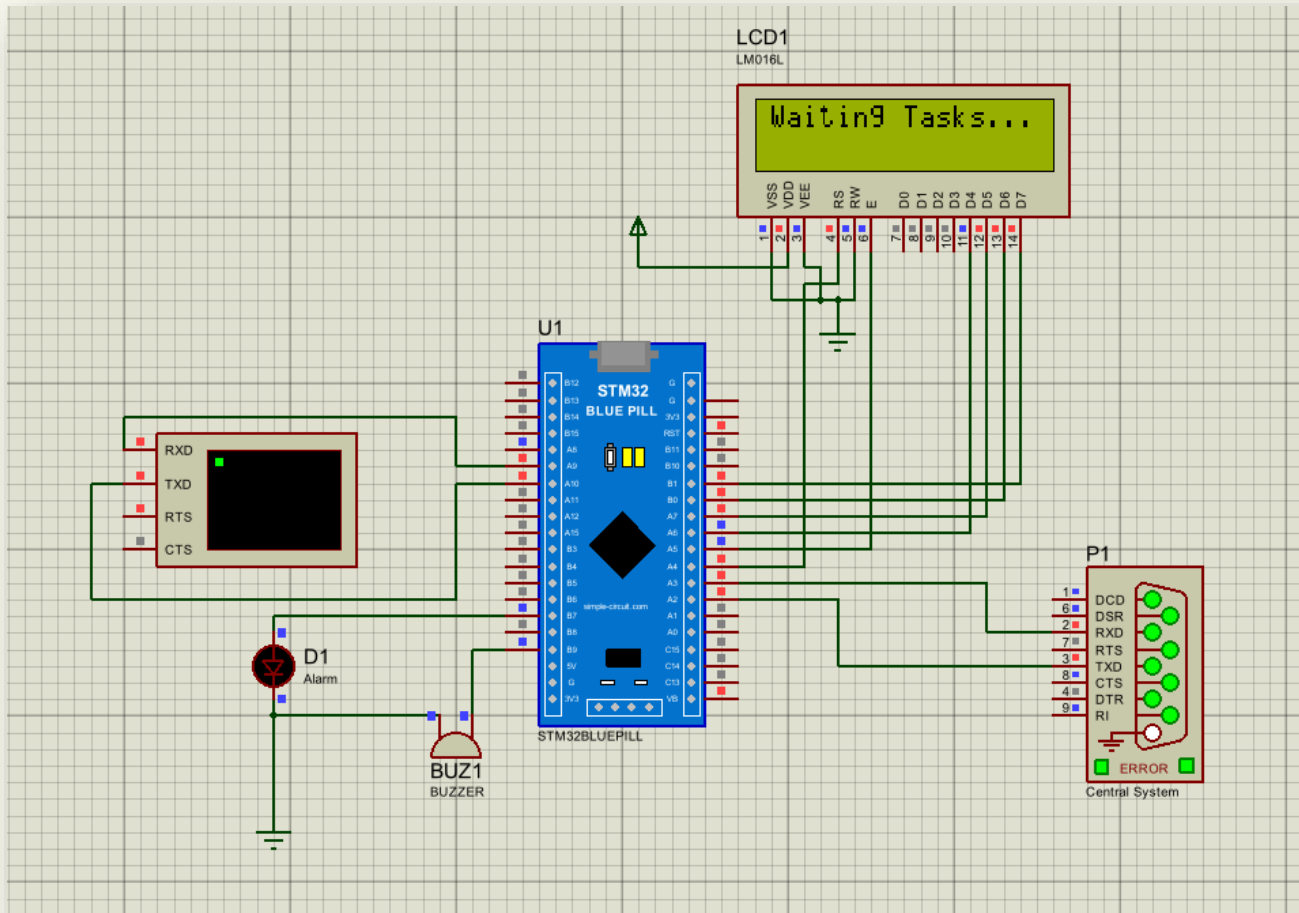
۲. سرعت پردازش و مسیر دهی دیتابیس

- اعمال فرمول باقی مانده تقسیم بر 1000 بر روی ID کارگر برای استخراج شماره خط هدف و پرش مستقیم به داده (جهت جلوگیری از کندی جستجوی خط به خط)
- پیاده سازی سیستم مسیردهی پویا با تعریف یک مسیر ریشه (DB_PATH_BASE)

۳. مدیریت حافظه و جلوگیری از تداخل داده‌ها

- • برای رفع مشکل سرریز بافر (Buffer Overflow) مرز آرایه ها به دقت کنترل شد و از توابع امنی نظیر strncpy استفاده گردید
- پیاده سازی مکانیزم "کپی سایه" در فایل های متنی برای جلوگیری از Overwrite شدن بایت ها و تداخل حافظه در دیتابیس

زیر بخش ۴؛ شبیه سازی و پیاده سازی سخت افزاری



- سیستم مرکزی: وظیفه دریافت، تحلیل و مدیریت داده های ایمنی (مثل خطا های مربوط به عدم پوشش مناسب یا رعایت نکردن فاصله ایمنی) را بر عهده دارد. این سیستم همچنین مجهز به یک مکانیزم پاداش دهی (bonus) است که با پایش مداوم عملکرد کارگران، پاداش انجام کار های اضافی را محاسبه و به صورت خودکار در دیتابیس و سوابق آنها ثبت می کند.
- نمایشگر: وضعیت لحظه ای سیستم را به صورت دیداری گزارش می دهد. در زمان بروز حادثه وضعیت هشدار (رنگ قرمز) را به نمایش می گذارد و پس از رفع مشکل، با نمایش پیام "Error Solved!" کارگر را مطلع می کند
- LED (چراغ خطر): به عنوان یک هشدار دهنده بصری سریع عمل کرده و به محض تشخیص هرگونه نقض در پروتکل های ایمنی، فوراً روشن می شود
- آژیر: نقش هشدار دهنده صوتی سیستم را ایفا می کند و همزمان با LED و تغییر وضعیت نمایشگر، در کسری از ثانیه به صدا در می آید تا توجه افراد حاضر در محیط را به سرعت جلب کند.

```
=====
Smart Factory - Generation-Fair Central System
=====
[1] Create & Inject New Extra Task
[2] Clear Screen
[0] Exit System
=====
Choose an option or wait for incoming serial data: |
```

پایش دستگاه های تولیدی

کنترل ویبراسیون ، تحلیل و نمایش در المان های بصری

تحلیل بر اساس استاندارد های ISO10816_3



زیر بخش ۵؛ از مهندسی تا صنعت؛

تبدیل یک ایده‌ی مهندسی به یک سیستم مانیتورینگ صنعتی در کارخانه‌های هوشمند، یکی از مهم‌ترین روش‌های جلوگیری از خرابی تجهیزات، پایش وضعیت ماشین آلات Condition Monitoring است. لرزش غیرعادی در تجهیزاتی مانند موتور، پمپ و کمپرسور می‌تواند نشانه مشکلاتی مانند:

- خرابی قطعات مکانیکی

- عدم بالانس

- فرسودگی تجهیزات

باشد.

در این پروژه یک نمونه بسیار ساده از یک سیستم صنعتی طراحی شد که:

- مقدار لرزش را دریافت می‌کند.

- داده را توسط میکروکنترلر STM32 پردازش می‌کند.

- وضعیت سلامت دستگاه را تحلیل می‌کند.

- نتیجه را به صورت بصری و متنی نمایش می‌دهد.

هدف نهایی این ایده، حرکت از نگهداری واکنشی به سمت نگهداری پیش‌بینانه در کارخانه‌های هوشمند است.

زیر بخش ۵؛ از مهندسی تا صنعت؛

جدول مرزهای استاندارد ISO 10816-3 برای ماشین آلات صنعتی متوسط
مانند موتور، پمپ و کمپرسور

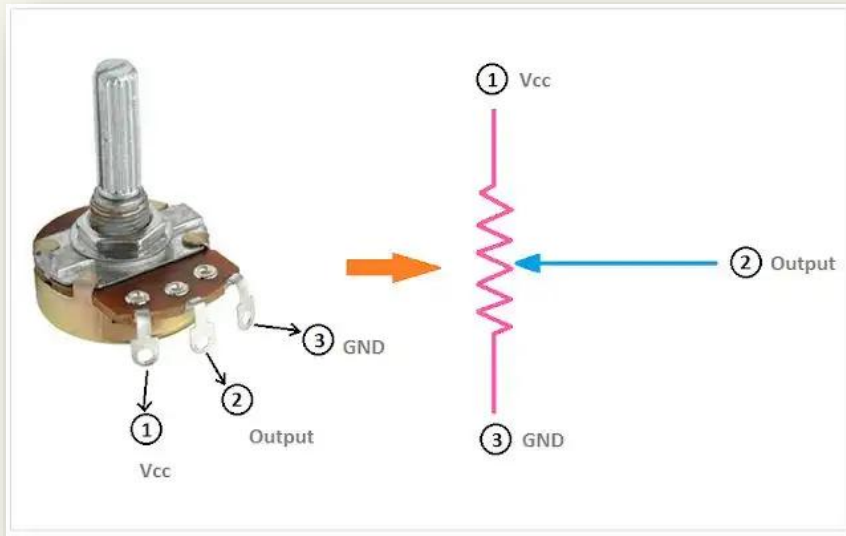
ISO 10816-3		Group 1		Group 2	
		Large machines 300 kW < power < 50 MW		Medium machines 15 kW < power < 300 kW	
in/sec peak	mm/sec rms	Motor height > 315 mm		Motor 160 mm < height < 315 mm	
0.61	11.0		Damage occurs		
0.39	7.1				
0.25	4.5		Restricted operation		
0.19	3.5				
0.16	2.8				
0.13	2.3		Unrestricted operation		
0.08	1.4				
0.04	0.7	Newly commissioned machinery			
0.00	0.0				
Foundation		Rigid	Flexible	Rigid	Flexible

زیر بخش ۵؛ شبیه سازی محصول؛

در سیستم واقعی از سنسورهای لرزش استفاده می‌شود اما برای ساخت نمونه آزمایشگاهی، از یک پتانسیومتر به عنوان ورودی آنالوگ قابل کنترل استفاده شد.

ایده‌ی شبیه سازی:

تغییر مقدار پتانسیومتر: تغییر ولتاژ ورودی ADC
مقدار: ADC نشان دهنده شدت لرزش ماشین
افزایش مقدار ورودی: افزایش سطح هشدار
به این ترتیب شرایط مختلف یک ماشین از حالت سالم تا وضعیت خطرناک شبیه‌سازی شد.





HAL (adc, gpio, usart, rtc)

ADC خواندن داده
GPIO کنترل LED برای نمایش وضعیت و دریافت فرمان
USART مانیتورینگ لحظه ای



نمایشگر ال سی دی

درایور اختصاصی LCD در حالت 4-bit که امکانات زیر را فراهم می‌کند:
مقدار دهی اولیه LCD
ارسال فرمان و داده

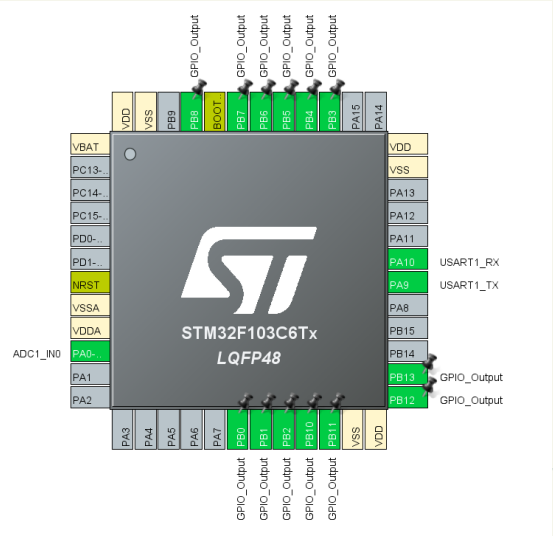


پیاده سازی هسته ارتباطی و پردازش خطا

وظایف اصلی پردازش داده های سنسور که وظایف زیر را انجام می دهند:
خواندن مقدار ADC
تبدیل به مقادیر لرزش mm/s
تعیین وضعیت دستگاه
بروزرسانی LED
گزارش با UART

زیر بخش ۵؛ منطق سخت افزاری و پین ها

توضیحات و قطعه متصل شده	نوع عملکرد	نام سیگنال	پورت / پین
اتصال پتانسیومتر شبیه ساز سنسور لرزش ماشین آلات و تبدیل مقدار آنالوگ به سطح لرزش	ورودی آنالوگ	VIBRATION_SENSOR	PA0
اتصال رابط سریال برای ارسال اطلاعات وضعیت ماشین، مقدار لرزش و پیام های مانیتورینگ	ارتباط سریال	USART1	PA9 / PA10
کنترل LED های نمایش وضعیت ماشین شامل حالت عادی، هشدار و خطر	خروجی دیجیتال	STATUS_LED	PB0 / PB1
اتصال LCD 16x2 در حالت برای نمایش مقدار لرزش، واحد mm/s و وضعیت سیستم	خروجی دیجیتال	LCD_PARALLEL_BUS	PB3/PB4/PB5/PB6/PB7/PB8
کنترل نوار LED نمایش شدت لرزش Trend Bar جهت نمایش سطح افزایش ارتعاش	خروجی دیجیتال	VIBRATION_LEVEL_BAR	PB10 / PB11 / PB12 / PB13



زیر بخش ۵؛ منطق عملکرد و الگوریتم‌های اجرایی

۱. فیلتر سازی و پایداری داده ها

سیگنال شبیه‌سازی شده سنسور لرزش از طریق ورودی آنالوگ PA0 / ADC1_IN0 توسط مبدل ADC داخلی STM32 دریافت می‌شود. مقدار خام ADC در بازه ۰ تا ۴۰۹۵ (رزولوشن ۱۲ بیتی) خوانده شده و با تبدیل خطی به مقدار شدت لرزش بر حسب mm/s تبدیل می‌گردد.

برای کاهش نویز موجود در نمونه‌برداری ADC و جلوگیری از تغییرات لحظه‌ای وضعیت ماشین، داده‌های دریافت شده پردازش می‌شوند. مقدار نهایی لرزش پس از تبدیل ADC به عنوان مقدار معتبر سیستم برای تصمیم‌گیری‌های بعدی استفاده می‌شود.

۲. مقایسه با آستانه های استاندارد لرزش

مقدار لرزش محاسبه شده با محدوده‌های استاندارد ISO 10816-3 مقایسه می‌شود:

اگر مقدار لرزش بین ۰ تا ۲,۸ mm/s باشد:
وضعیت ماشین NORMAL تشخیص داده می‌شود.
LED سبز روشن باقی می‌ماند.

اگر مقدار لرزش بین ۲,۸ تا ۷,۱ mm/s باشد:
وضعیت ماشین WARNING تشخیص داده می‌شود.
LED قرمز با چشمک آهسته فعال می‌شود.

اگر مقدار لرزش بیشتر از ۷,۱ mm/s باشد:
وضعیت ماشین DANGER تشخیص داده می‌شود.
LED قرمز با سرعت بالا چشمک می‌زند.

زیر بخش ۵؛ منطق عملکرد و الگوریتم‌های اجرایی

۳. منطق هشدار لرزش و مدیریت وضعیت اضطراری ماشین

ورودی سنسور لرزش شبیه‌سازی شده از طریق پایه PA0 / ADC1_IN0 به صورت پیوسته نمونه‌برداری می‌شود.

مقدار لرزش پس از تبدیل ADC به واحد mm/s با محدوده‌های استاندارد مقایسه شده و سطح سلامت ماشین تعیین می‌گردد.

• فعال‌سازی حالت هشدار:

در صورت عبور مقدار لرزش از محدوده مجاز، سیستم وارد وضعیت هشدار می‌شود:

در محدوده 2.8 تا 7.1 WARNING

وضعیت هشدار فعال می‌شود.

LED قرمز با سرعت کم چشمک می‌زند.

در محدوده بیشتر DANGER از 7.1

وضعیت خطر فعال می‌شود.

LED قرمز با سرعت بالا چشمک می‌زند.

۴. نمایش سطح لرزش 4 مرحله ای

برای نمایش بصری شدت لرزش، مقدار لرزش پردازش شده به چهار سطح مساوی تقسیم شده و توسط چهار LED مستقل نمایش داده می‌شود. هر LED نشان‌دهنده افزایش یک مرحله از شدت ارتعاش بوده و با افزایش مقدار لرزش، تعداد LED های روشن بیشتر می‌شود.

تقسیم‌بندی: Trend Bar:

LED1 سطح اول شدت لرزش (مقادیر پایین)

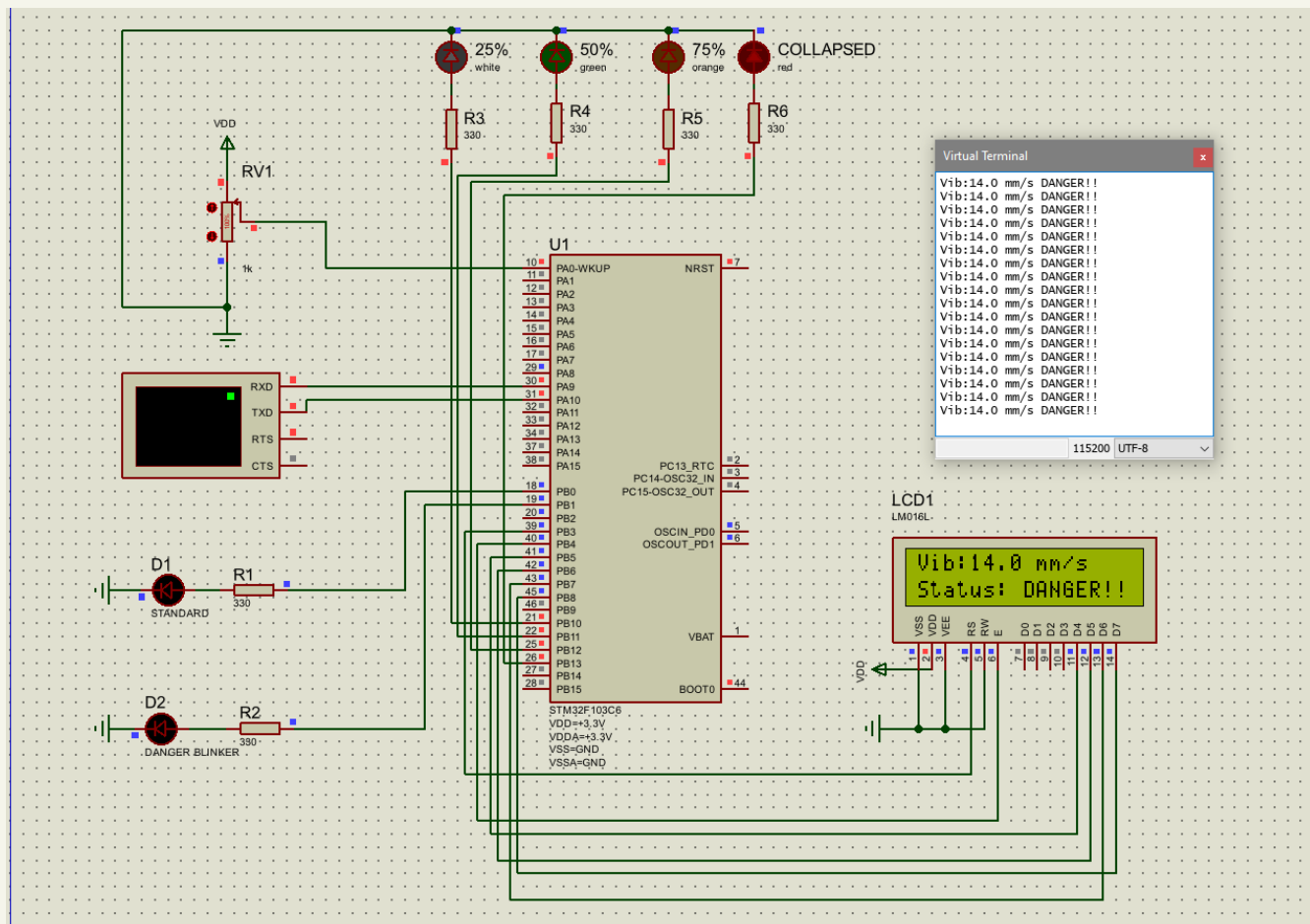
LED2 سطح دوم شدت لرزش (افزایش متوسط ارتعاش)

LED3 سطح سوم شدت لرزش (نزدیک شدن به محدوده هشدار)

LED4 سطح چهارم شدت لرزش (بیشترین سطح ارتعاش)

این نمایشگر مستقل از وضعیت هشدار عمل کرده و برای مشاهده روند افزایش یا کاهش لرزش ماشین در لحظه استفاده می‌شود.

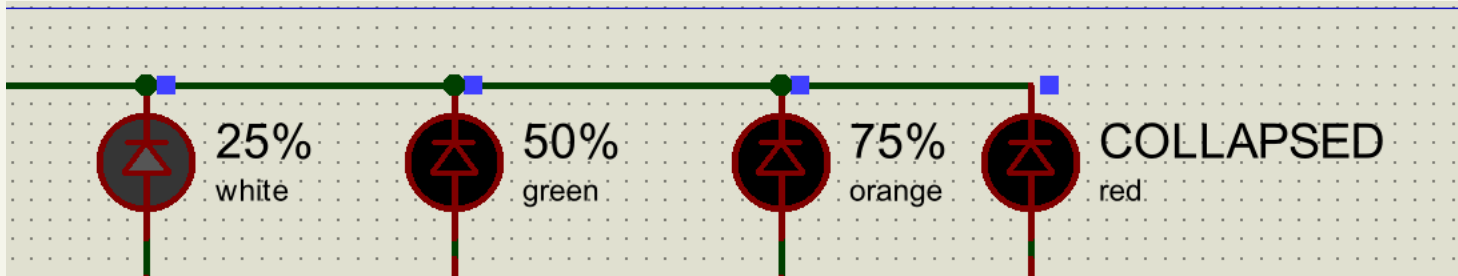
زیر بخش ۵؛ شبیه سازی و پیاده سازی سخت افزاری



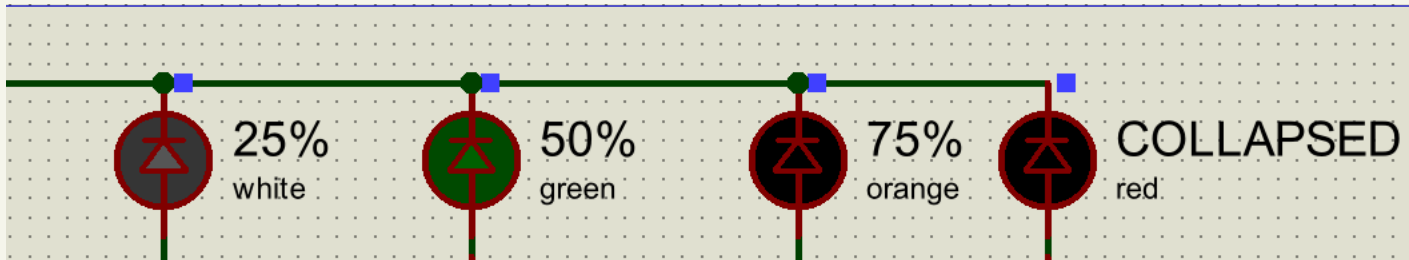
- میکروکنترلر: STM32F103 هسته‌ای اصلی پردازش سیستم که وظیفه‌ی دریافت داده‌ی لرزش، تحلیل وضعیت ماشین و کنترل خروجی‌ها را بر عهده دارد.
- ورودی شبیه‌ساز لرزش: Potentiometer: متصل به پایه PA0 (ADC Channel 0) جهت تولید سیگنال آنالوگ مشابه سنسور لرزش و شبیه‌سازی شرایط مختلف ماشین.
- نمایشگر: LCD 16x2: نمایش مقدار لحظه‌ای لرزش بر حسب mm/s وضعیت عملکرد ماشین DANGER، WARNING، NORMAL برای ارتباط با اپراتور.
- نشانگرهای وضعیت: LED سبز: نمایش عملکرد عادی ماشین
- LED قرمز: نمایش شرایط هشدار و خطر با الگوی چشمک متفاوت
- نوار شدت لرزش با 4 ال ای دی: نمایش گرافیکی سطح لرزش به صورت پلکانی، برای تشخیص سریع شدت ارتعاش بدون نیاز به بررسی مقدار عددی.

زیر بخش ۵؛ شبیه سازی و پیاده سازی سخت افزاری

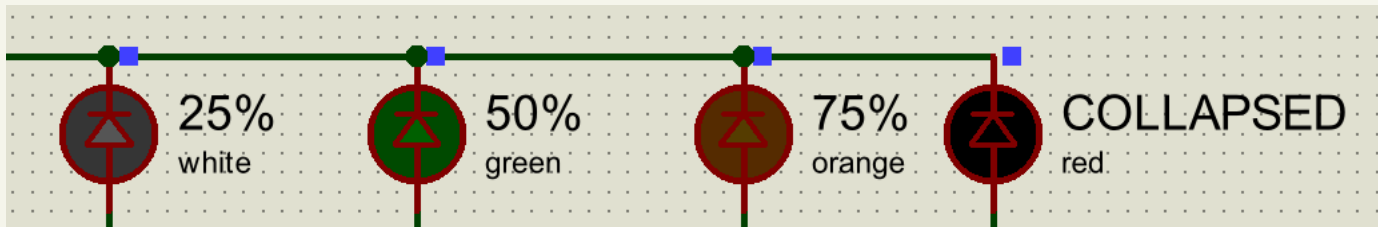
مراحل 4 گانه از
استاندارد تا فروپاشی



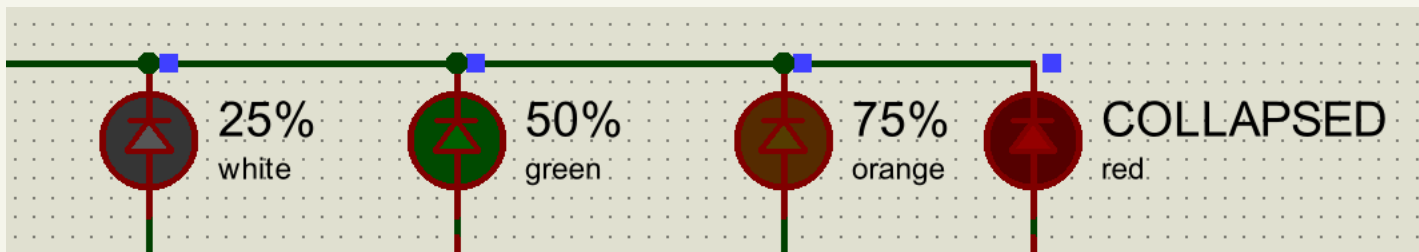
0 – 25 %



26 – 50 %



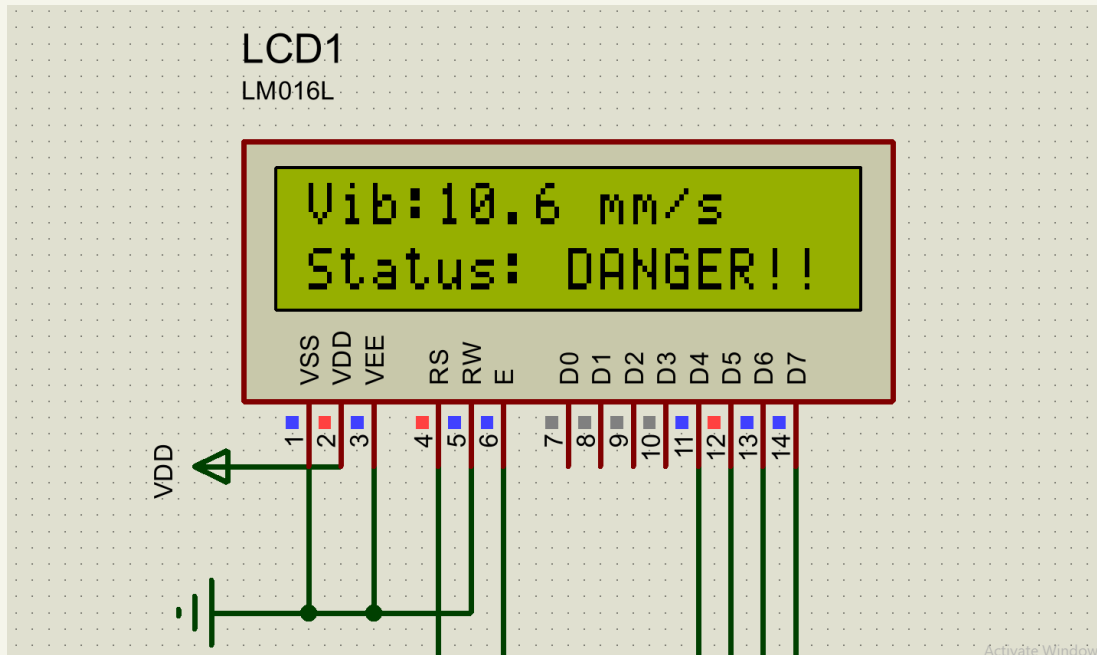
51 – 75 %



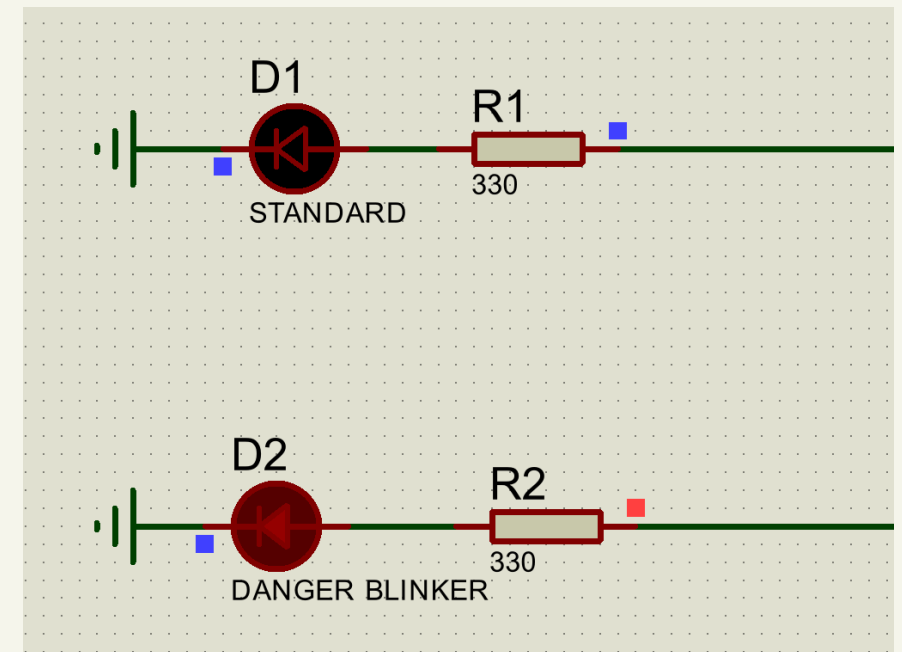
76 – 100 %

زیر بخش ۵؛ شبیه سازی و پیاده سازی سخت افزاری

خروجی در یک ال سی دی ۱۶ * ۲



- سرعت چشمک متغیر ال ای دی خطر است
- ال ای دی استاندارد در وضعیت خود استاندارد خواهد ماند



ارتباط با پایتون

رابط کاری پنل پایتون

ارتباط با پایتون؛ ارتباط CAM/MCU

دو پینل پایتون جدا وجود دارد که هرکدام نقش متفاوتی دارند؛

پینل گرافیکی:

داشبورد گرافیکی چهاربخشی در نظر گرفته شده که از طریق پورت سریال COM8، (9600 baud rate) مستقیماً به برد STM32 وصل می‌شود. برد داده‌های تله‌متری را با فرمت متنی خاص خودش

TIME=...|CUR=...|FIRE=1|LOCK=0

روی UART

می‌فرستد، و یک ترد در پایتون serial_rx_thread این خط‌ها را می‌خواند،

پارس می‌کند و هر ۱۵۰ میلی‌ثانیه رابط کاربری را به‌روزرسانی می‌کند.

علت استفاده از پایتون ؟

STM32 هیچ قابلیت رندر گرافیکی ندارد و فقط از طریق UART می‌تواند داده خام بفرستد؛

پایتون این داده‌ی خام را به یک داشبورد بصری زنده (دما، وضعیت آژیر، قفل ایمنی) تبدیل می‌کند.

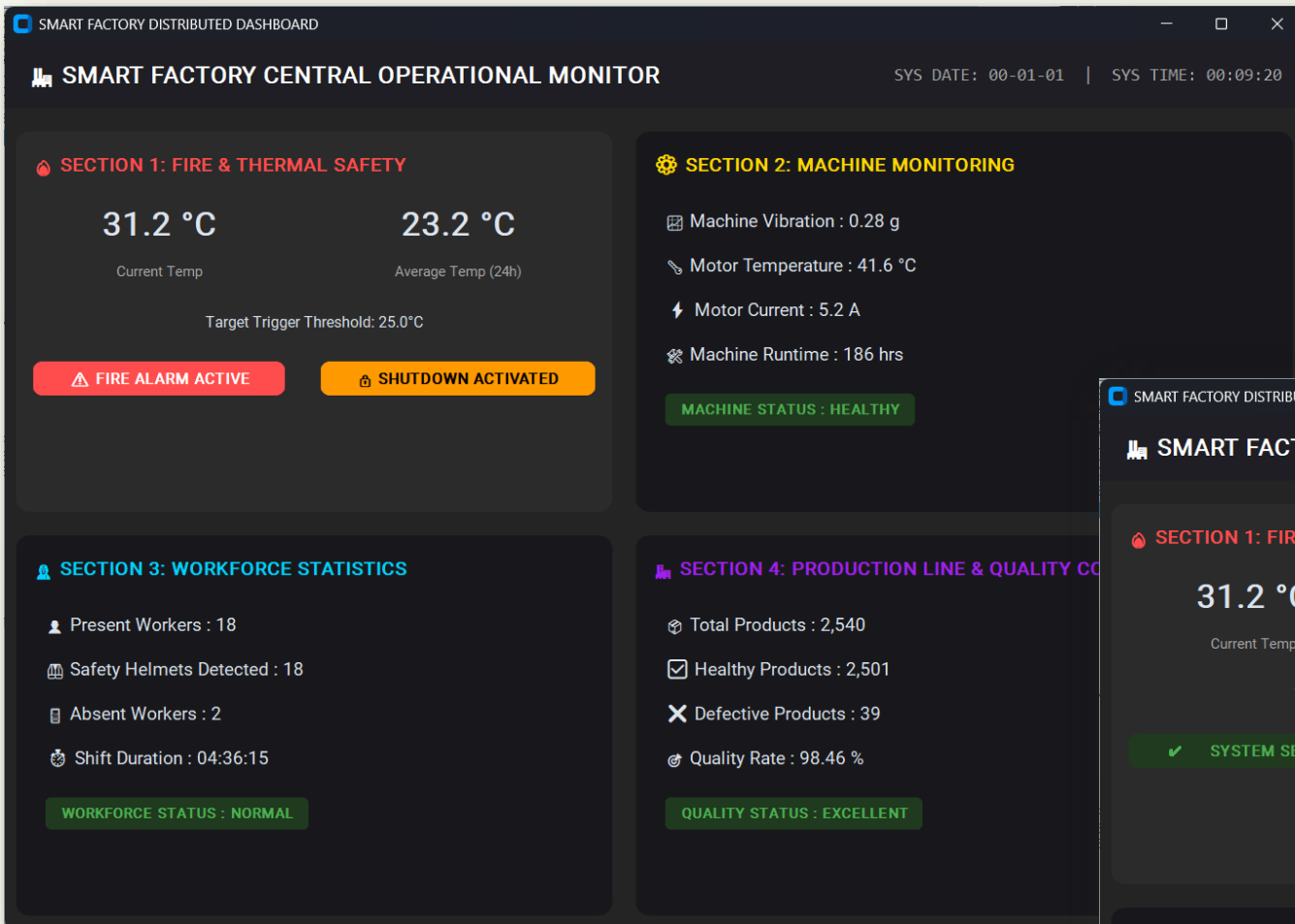
ارتباط با پایتون؛ ارتباط CAM/MCU

دو پنل پایتون جدا وجود دارد که هرکدام نقش متفاوتی دارند؛

سرور(سرور دوطرفه + دیتابیس فایل):

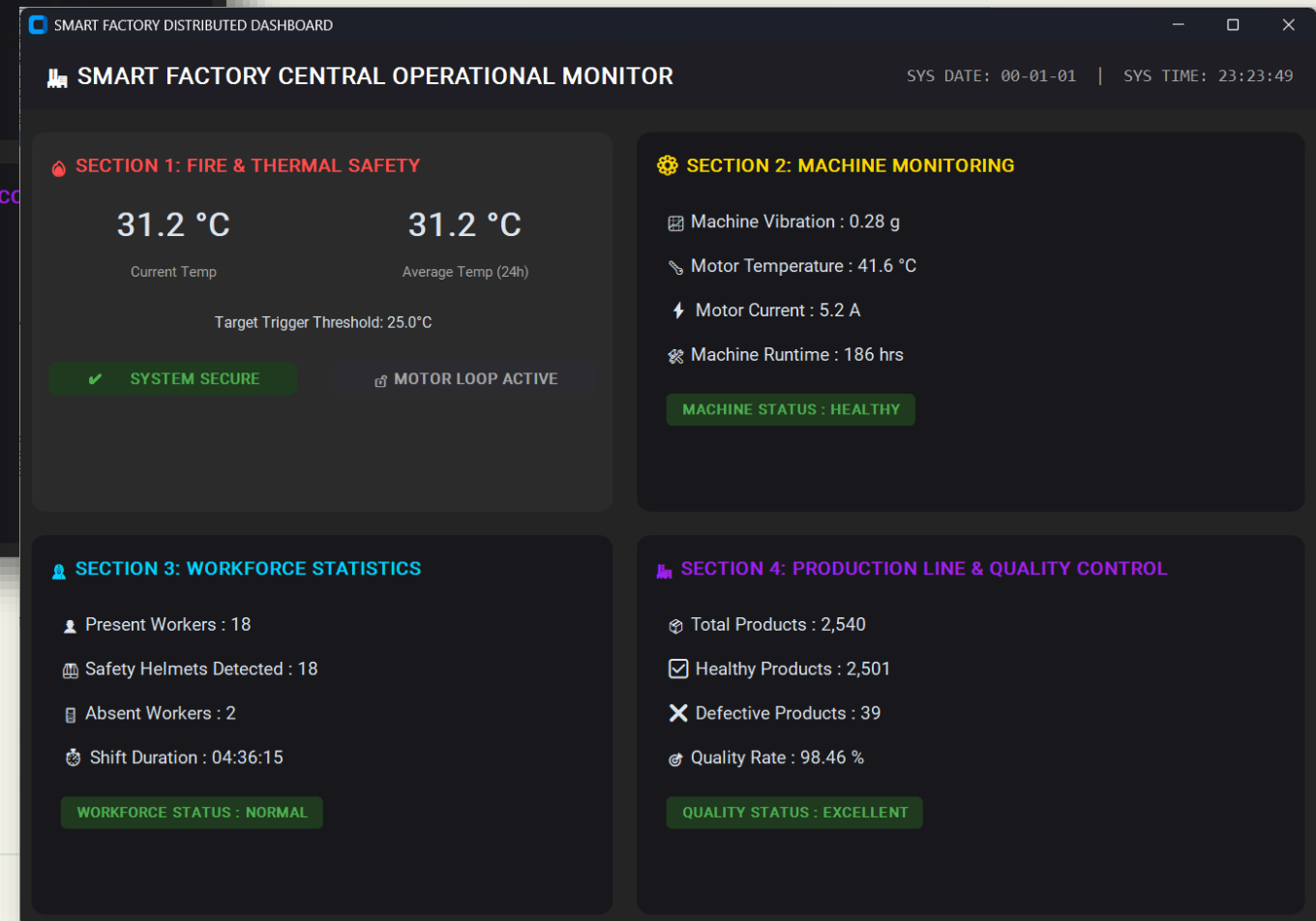
این اسکریپت روی پورت دیگری COM2 به بورد مدیریت نیروی کار وصل می‌شود و یک ارتباط دوطرفه برقرار می‌کند. وقتی MCU پیام‌هایی مثل REQ_NAME:, CHECKIN:, CHECKOUT:, VACATION: یا MANAGER:INFO: می‌فرستد، پایتون آن را پارس کرده، در فایل‌های متنی که نقش یک دیتابیس ساده را دارند (worker_ID.txt، پوشه‌های حضور/غیاب/مرخصی) جست‌وجو یا ثبت می‌کند، و نتیجه (نام کارگر، پیام موفقیت/خطا) را دوباره از طریق سریال به MCU برمی‌گرداند تا روی LCD نمایش داده شود. علت این معماری؟

STM32F103 نه فایل سیستمی دارد و نه قدرت پردازشی برای نگه‌داشتن دیتابیس چندصد نفره‌ی کارگران؛ بنابراین منطق سنگین (جست‌وجو، ذخیره‌سازی دائمی، تاریخچه) به لایه‌ی پایتون روی کامپیوتر منتقل شده، درحالی‌که منطق حیاتی و بلادرنگ (تشخیص لمس انگشت، کنترل LCD، هشدار فوری) همچنان روی خود میکروکنترلر باقی مانده تا سرعت و قابلیت اطمینان real-time حفظ شود.



ارتباط با پایتون؛ ارتباط CAM/MCU

رابط گرافیکی این اپلیکیشن به شکل زیر است؛



چالش‌ها و ادامه توسعه

ایده‌ها و تکمیل پروژه

۱. سیستم آتش و ایمنی توقف LCD و UART پس از دریافت دستور: انتقال پردازش دستورات از وقفه UART به حلقه اصلی برنامه. ثب نادرست مقدار دما: پیاده‌سازی تابع اختصاصی تبدیل رشته به عدد.
 ۲. مدیریت نیروی کارسریز بافر در ارتباط سریال: کنترل طول ورودی و استفاده از توابع امن‌تر. خطای: Linker (Undefined Symbol) بازنویسی و جایگذاری صحیح توابع حذف‌شده. خراب شدن فایل دیتابیس: استفاده از روش Shadow Copying برای ذخیره‌سازی امن.
 ۳. کنترل کیفی عدم عملکرد: ADC شناسایی خرابی سخت‌افزار و تعویض برد. ناپایداری تشخیص محصول: بازیابی و ساده‌سازی منطق تصمیم‌گیری.
 ۴. مانیتورینگ دستگاه‌های تولیدی خطای اتصال VCC و GND در Proteus: تنظیم دسقی پایه‌های تغذیه. نمایش اعداد نادرست روی LCD: افزایش اندازه بافر و اصلاح محاسبات.
- افت سرعت مانیتورینگ دو دستگاه: حفظ نسخه پایدار تک‌دستگاهی به دلیل محدودیت Proteus. عدم پخش صدای BUZZER: تأیید عملکرد هشدار از طریق بررسی سیگنال خروجی.

Embedded AI?

۱. سیستم آتش و ایمنی گسترش به بیش از دو ناحیه‌ی حریق تشخیص دود/آتش با پردازش تصویر مبتنی بر هوش مصنوعی و دوربین به‌جای فقط حسگر دیجیتال
۲. مدیریت نیروی کارگردش کار تأیید انجام تسک تعمیر سیستم مرکزی به حسابداری و حقوق و دستمزد تشخیص وضعیت لباس ایمنی و فاصله‌ی کارگر با پردازش تصویر (دوربین)
۳. کنترل کیفیت شخیص عیوب ظاهری با پردازش تصویر / بینایی ماشین به‌جای فقط وزن، طول و رنگ استفاده از هوش مصنوعی (مدل‌های یادگیری ماشین) برای دسته‌بندی هوشمندتر محصول سالم/معیوب
۴. مانیتورینگ دستگاه‌های تولیدی مانیتورینگ هم‌زمان چند دستگاه (چند کاناله) سرویس دوره‌ای بر پایه‌ی ساعت کارکرد (نگهداری پیشگیرانه) هشدار صوتی واقعی با ترانزیستور درایور پیاده‌سازی کامل روی سخت‌افزار واقعی تشخیص هوشمندانه‌ی الگوی خرابی با هوش مصنوعی (Predictive Maintenance مبتنی بر داده)
۵. توسعه‌های زیرساختی مشترک (کل پروژه) جایگزینی ارتباط سیمی با WiFi یا بلوتوث مثلاً مازول ESP32 برای مقیاس‌پذیری در سالن تولید استفاده از بردهای قوی‌تر STM32F4 یا ESP32 برای پردازش هم‌زمان تصویر، AI و چند سنسور جایگزینی دیتابیس فایلی با یک دیتابیس واقعی افزودن احراز هویت واقعی به‌جای فرمان‌های متنی ساده یکپارچه سازی چهار زیرسیستم در یک داشبورد و سرور مرکزی واحد

نتیجه گیری و منابع

منابع و سورس کدها



در این پروژه تلاش شد یک نمونه کاربردی از مفهوم کارخانه هوشمند در مقیاس آزمایشگاهی طراحی و پیاده سازی شود؛ سیستمی که چهار زیرسیستم مانیتورینگ ماشین آلات، اعلام و اطفاء حریق، کنترل کیفیت و مدیریت نیروی کار را در قالب یک معماری سه لایه (فریمور/سخت افزار، سیستم مرکزی و پایگاه داده، و اپلیکیشن کاربری پایتون) به صورت یکپارچه کنار هم قرار میدهد.

منابع

STM32F103x6/x8 Datasheet, STMicroelectronics
STM32 HAL Driver User Manual (UM1850), STMicroelectronics
Hitachi HD44780U LCD Controller/Driver Datasheet
ISO 10816-3: Mechanical Vibration — Evaluation of Machine Vibration by Measurements on Non-Rotating
Proteus Professional 9 Documentation, Labcenter Electronics
Keil MDK-ARM / ARM Compiler V6 Documentation, Keil (an ARM Company)
STM32IDE and STMCubeMX
مستندات فنی و گزارش های داخلی پروژه ی کارخانه ی هوشمند



https://github.com/alznvd6/SMART_FACTORY/